12.LoRA

18 March 2026 12:57

Large scale pretraining on general domain data
↳ then adaptation to particular tasks

As we pretrain large models → full fine-tuning, which retains all parameters becomes less feasible

GPT-3 is 175B → deploying fine tuned models with all 175B parameters
↓
highly expensive

Low Rank Adaptation

↓
→ freezes the pretrained model weights
→ injects trainable rank decomposition matrices into each layer of the transformer architecture
↓
reduces the number of trainable parameters for downstream tasks

Compared to GPT3 (175B) fine tuned with Adam
LoRA reduces parameters by 10,000 times & gpu memory req by 3 times

performs on par or better than fine tuning in model quality on Roberta, gpt2, gpt3 etc
despite having fewer trainable parameters,
a higher training throughput,
& unlike adapters, no additional inference latency

paper says typical training

two stages : Pretraining → Train on massive datasets
" fine tuning → Adapt pretrained model to a specific task
entire model param's are updated

gets expensive

Variants → full fine tuning → all params updated
→ partial fine tuning → some layer
→ parameter efficient metho → some added modules

Many tasks → you must store full copy of the model per task → very expensive

Before LoRA people tried Adapter, prefix tuning, prompt tuning
These methods freeze pretrained wts
Add some new modules
train only new modules

eg Adapter
instead of changing the transformer layer
x → Linear → attention
they insert
x → Linear → Adapter → attention
This adds extra layers

Adapters introduce extra inference latency
Because the model becomes deeper

LoRA injects trainable rank decomposition matrices into each layer

We have W_q, W_k, W_k, W_o } large matrices

$W \in R^{(d \times d)}$

for GPT-3 $d \approx 12288$

Matrix size (12288×12288)

During normal fine tuning

$W_{new} = W + \Delta W$ ΔW = learned wt update

you train every entry of ΔW

Authors here say that this ΔW actually lies in low rank space
↓
meaning even though ΔW is large (12288×12288)

→ Authors here say that this ΔW actually lies in low rank space

↓

meaning even though ΔW is large (12288×12288)

the Useful update is much simpler

it can be approximated as $\Delta W \approx AB$

$A \in R^{\wedge(d \times r)}$ $B \in R^{\wedge(r \times d)}$

So instead of learning $12288 \times 12288 \approx 150M$ params

we learn $12288 \times 2 + 2 \times 12288 \approx 50K$ params

$(r = 2$ eg$)$

During LoRA w is frozen A & B are trained

So the layer becomes

$$\text{Output} = Wx + BAx = (W + BA)x$$

paper claim Updates for large models live in a low intrinsic dimension

The parameter space is huge 175B

The actual useful directions for task adaptation are very small

eg 3D space (x, y, z)

but if required solutions lie on 2D plane (x, y)

the intrinsic dimension $= 2$

Weight update space = huge

Useful update directions = Few

Reduced Usable Seq length, some ported

→ GPU memory fixed

→ Adapter adds params

→ memory, resources

→ more params less room for KV cache

→ shorter context length

LoRA → higher throughput

↳ no of training steps per second

→ fewer params

→ fewer gradients

→ less memory

→ less optimizer state

LoRA is orthogonal to other methods means → it is indepent & compatible with other meths

LoRA assumes ΔW to be low rank

It doesn't force an arbitrary matrix to become low rank

↳ assumes useful adaptation directions are already low rank

if not → model cannot express the needed update

→ adaptation quality drops

→ LoRA would fail to match full fine-tuning

$\Delta W \in \mathbb{R}^{D \times D}$

↳ $A \in \mathbb{R}^{D \times r}, B \in \mathbb{R}^{r \times D}$

$\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$

$\text{rank}(A) \leq r$ $\text{rank}(B) \leq r$

A has only r columns B only r rows

$\text{rank}(AB) \leq r$

Matrix B does

$R^{\wedge}D \rightarrow R^{\wedge}r$

it compresses the representation to an r-dimensional subspace

Then Matrix A does

$R^{\wedge}r \rightarrow R^{\wedge}D$

it projects back into the original space

Update can only live in an r dimensional subspace of the full D-dimensional space

instead of allowing updates in all directions $\Delta W \in R^{\wedge}(D \times D)$ LoRA constrains updates to $\Delta W \in \{\text{rank} \leq r \text{ matrices}\}$

So the optimizer searches in a very small subspace of possible updates

pretrained Autoregressive Model $P_\phi(y|x)$ parametrized by ϕ

pretrained Autoregressive Model $P_\phi(y|x)$ parametrized by ϕ

training dataset $Z = \{(x_i, y_i)\}_{i=1}^{N}$ x_i, y_i are sequences of tokens

query target

During full fine tuning model is initialized to pre trained wts ϕ_0

and updated to $\phi_0 + \Delta\phi$

by repeatedly following the gradient

to maximize $\max_{\phi} \sum_{(x,y)\in Z} \sum_{t=1}^{|y|} \log(P_\phi(y_t|x, y_{<t}))$

$\Delta\phi$ dimension equals $|\phi_0|$

Parameter efficient approach

instead of optimizing $\Delta\phi$ ← $\Delta\phi = \Delta\phi(\Theta)$ is further encoded by a much smaller-sized of parameters Θ with $|\Theta| << |\phi_0|$

it is produced from a smaller parameter set Θ

Thus the task of finding $\Delta\phi$ becomes optimizing over Θ

$$\max_{\Theta} \sum_{(x,y)\in Z} \sum_{t=1}^{|y|} \log(P_{\phi_0} + \Delta\phi(\Theta)(y_t|x, y_{<t}))$$

Two prior directions

1. Adapter layers - Insert extra layers inside transformer blocks

Structure Transformer layer → Adapter → next layer Extra sequential compute → Inference latency

2. Prompt/prefix tuning

↳ It learns continuous vectors that are prepended to the attention keys & values

Model sees [prefix vectors] + [input tokens]

↓

trainable parameters

prefix occupies sequence length → reduces usable context window

$W_0 \in \mathbb{R}^{d\times k}$ $h = W_0 x + BAx$

$W = W_0 + \Delta W$ input x → low rank vectors

$W = W_0 + BA$ A project to r dim

$B \in \mathbb{R}^{d\times r}$ ↓

$A \in \mathbb{R}^{r\times k}$ B project back to d-dim

Initialization $A \sim N(0, \sigma^2)$

$B = 0$

So initially $\Delta W = BA = 0$

Model starts identical to the pretrained model

Very important for stability

$h_0 = W_0 x + (\alpha/r) BA$ (Scaling)

↓

why? i) control update magnitude

without scaling, magnitudes of updates depend on r

If you increase r, update magnitude grows

α/r keeps the expected update size roughly constant when r changes

ii) Decouple rank from learning rate

Otherwise whenever you change 'r' you need to retune learning rate

Scaling ensures rank change ≠ Optimizer retuning

They chose $\alpha = r$, $\alpha/r = 1$

Inference cost is identical to full fine-tuning

problem Different tasks → Different Lora Matrices

But batching requires same weights for the batch

So serving thousands of Lora adapters requires

Inference cost is identical to full fine-tuning

$W' = W_0 + (\alpha/r) BA$

$h = W'x$ no extra computation graph

But batching requires same weights for the batch

So mixing tasks in one-batch is hard after merging

Workaround → (no merging wts
Choose LoRA wts dynamically
→ Introduces latency

Most adaptation happens in Attention projections

1) Given a parameter budget constraint, which subset of weight matrices in a pre-trained transformer we should adapt to maximize downstream performance

2) Is the "optimal" adaptation matrix ΔW really rank deficient?
If so what is a good rank in practice?

3) What is the connection between ΔW and W?
Does ΔW highly correlate with W?
How large is ΔW comparing to W?

① we only consider wt matrices in attention module

Adapting more matrices with low rank > Adapting one matrix with high rank

Suggests Adaptation signal is distributed

② is ΔW truly low rank

If lora works with smaller, it implies → true ΔW lives in a small sub space

③ How does ΔW relate to W?

Does LoRA create new directions or amplify existing directions in W

subspace similarity

ΔW

Do SVD $\Delta W = U \Sigma V^T$ → left singular vectors, → right singular vectors

These vectors represent the principal directions of the update

eg if lora uses rank = 8 then the update lives in span of top 8 singular vectors

8-dimensional subspace

They train two LoRA models
$A_{r=8}$ $A_{r=64}$ Both learn ΔW

Do the important directions learnt with $r=8$ appear inside directions learnt with $r=64$

↳ If yes → then small rank already captures the important adaptation directions

from heatmaps
they say
top singular vectors overlap strongly

$r=8$ already captures the most important update directions
↓ extra directions are noise or minor

$$\phi(A_{r=8}, A_{r=64}, i, j) = \frac{\| U^{i,T}_{A_{r=8}} U^{j}_{A_{r=64}} \|_F^2}{\min(i,j)} \in [0,1]$$

↓ projection overlap between singular vector sets

Simplified intuition
V_small = direction learnt with $r=8$
V_large = $r=64$
Compute $\cos(\text{angle}(V_{small}, V_{large}))$

Intuition for LoRA
$\Delta W = \sum_{i=1}^{r} (b_i a_i^T)$

LoRA learns r independent update directions

SVD recap
factorize any real/complex matrix into three simpler matrices

for any $m \times n$ matrix A, SVD is
A → Original $m \times n$ matrix
U → $m \times m$ orthogonal matrix (left singular vectors)
Σ → $m \times n$ diagonal matrix containing singular values
V^T → transpose of V, where V is $n \times n$ orthogonal matrix (Right singular vectors)

Orthogonal matrices satisfy $U^TU = I$ $V^TV = I$

Structure of Σ matrix
↓
contains singular values arranged in decreasing order

$\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq \sigma_4 \geq 0$

ex $\Sigma = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix}$

These values are square roots of the eigen values of A^TA

SVD describes how matrix transforms space

V^T rotates coordinate system
Σ stretches / compresses along axes
U rotates again to final orientation

So SVD represents any linear transformation as
Rotation → Scaling → Rotation

Steps to compute SVD
Compute A^TA
Find its eigen values & eigen vectors
Singular values $\sigma_i = \sqrt{\lambda_i}$
Eigen vectors of A^TA form
Compute $U = AV\Sigma^{-1}$
eg $A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$
$A = U\Sigma V^T$

PCA
Image compression
Recommender systems
Noise reduction
Solving Linear least squares problems

The most important update directions

↓ extra directions are noise or minor corrections

Strong evidence for low intrinsic rank

V_small = direction learnt with r=8

V_large = r=64

Compute cos(angle(V_small, V_large))

if 1 → same direction

0 → Orthogonal

So this measures alignment of learned directions

Solving Linear least squares problems

Grassmann manifold is simply the space of all k dimensional spaces

They use it to measure
↳ distance btwn two subspaces

(Instead of comparing vectors individually, they compare the spaces spanned by them)

They repeat the experiment with different random seeds

They compute subspace similarity Again

These matrices ΔW_q, ΔW_v (strong structure) Random gaussian (no structure)

learned directions are stable across runs

LoRA is not discovering random low rank updates

It consistently finds similar adaptation directions

Is ΔW aligned with the existing wt matrix W?

meaning → does adaptation modify directions already present in W?

$\Delta W = U \Sigma V^T$

project W onto those directions

SVD lets
finding principal update directions
Comparing subspaces
Measuring overlap between updates & pretrained weights

Without it there is no clear notion of direction in weight space

Compute $\| U^T W V \|_F$ → This measures how much of W lies in the ΔW subspace

Frobenius norm measures total energy of the matrix

↓

they check how much energy of W is aligned with ΔW directions

finding: ΔW strongly correlates with W

↳ LoRA isn't inventing new directions

ΔW amplifies what is already present in W

Task adaptation lives in low dimensional subspace

and

those directions already exist inside pretrained weights